GH-200 GitHub Actions Certification — 7-Day Action Plan

Exam Date: March 24, 2026 | Study Period: March 17–23

> ***How to use this guide:*** *Each day focuses on a specific GH-200 domain. Three hands-on projects are woven throughout the week — each project is implemented as a live workflow in this repository so you can* ***read the code, trigger the workflow, and watch it run*** *while you study.*

🗂️ GH-200 EXAM DOMAIN WEIGHTS

Domain	Weight
Author and maintain workflows	~40 %
Consume workflows (artifacts, caching, env vars)	~20 %
Author and maintain actions	~25 %
Manage GitHub Actions for the enterprise	~15 %

🚀 THREE HANDS-ON PROJECTS

Project	Focus	Workflow File
Project 1 — CI/CD Pipeline	Workflow syntax · events · jobs · matrix · caching · artifacts	`.github/workflows/project1-cicd-pipeline.yml`
Project 2 — Deployment Pipeline	Environments · secrets · approvals · concurrency · status badges	`.github/workflows/project2-deployment-pipeline.yml`
Project 3 — Reusable Workflows	Reusable workflows · composite actions	`.github/workflows/project3-reusable-workflows.yml`

& Custom Actions	· `workflow_call` · outputs	

Study the workflow file for the current day's project **before** reading the theory — seeing real YAML makes the concepts concrete.

📅 **DAY 1 — MONDAY, MARCH 17**

Topic: Workflow Fundamentals — Syntax, Events & Triggers

Exam weight: ~15 % | Project focus: Project 1

Core Concepts

1. Workflow File Location & Structure

Every GitHub Actions workflow lives in `.github/workflows/<name>.yml`.

```yaml
name: My Workflow          # display name (optional)

on:                        # trigger(s)
  push:
    branches: [main]

jobs:                      # one or more jobs
  my-job:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello!"
```

2. Trigger Events (`on`)

Event	When it fires
`push`	A commit is pushed to a branch
`pull_request`	PR opened, synchronised, or reopened
`workflow_dispatch`	Manual trigger from the UI or API
`schedule`	Cron expression (UTC)
`release`	A GitHub release is published

`workflow_call`	Called by another workflow
`repository_dispatch`	External system sends a POST to the GitHub API

```yaml
on:
  push:
    branches: [main, 'release/**']
    paths: ['src/**']           # only when src/ files chan
ge
  pull_request:
    types: [opened, synchronize]
  schedule:
    - cron: '0 8 * * 1'         # every Monday at 08:00 UTC
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target environment'
        required: true
        default: staging
        type: choice
        options: [staging, production]
```

3. Jobs

- Jobs run **in parallel** by default.
- Use `needs` to create a dependency chain (sequential execution).
- Each job gets a **fresh runner** (isolated environment).

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps: [ ... ]

  test:
    needs: build            # runs after build
    runs-on: ubuntu-latest
    steps: [ ... ]

  deploy:
    needs: [build, test]    # runs after BOTH
    runs-on: ubuntu-latest
    steps: [ ... ]
```

4. Steps

Steps run **sequentially** inside a job, sharing the same runner workspace.

```yaml
steps:
  - name: Checkout code
    uses: actions/checkout@v4         # use a published act
ion

  - name: Install dependencies
    run: npm ci                        # run a shell comman
d

  - name: Run tests
    run: npm test
    env:
      NODE_ENV: test                   # step-level env var
```

5. Matrix Strategy

Run the same job across multiple configurations automatically.

```yaml
strategy:
  matrix:
    node-version: [18.x, 20.x, 22.x]
    os: [ubuntu-latest, windows-latest]
  fail-fast: false    # don't cancel others if one fails
```

This creates `3 × 2 = 6` parallel jobs.

📋 ***Day 1 Quiz***

Q1. A workflow file must be placed in which directory?

- A) `.workflows/`
- B) `.github/actions/`
- C) `.github/workflows/` ✅
- D) `workflows/`

Q2. Which event triggers a workflow when a developer manually clicks "Run workflow" in the GitHub UI?

- A) `repository_dispatch`
- B) `workflow_dispatch` ✅
- C) `workflow_call`
- D) `manual`

Q3. Job B must run after Job A completes successfully. What keyword achieves this?

- A) `after`
- B) `requires`
- C) `depends_on`
- D) `needs` ✅

Q4. A matrix strategy with `os: [ubuntu, windows]` and `node: [18, 20, 22]` creates how many jobs?

- A) 2
- B) 3
- C) 5
- D) 6 ✅

Q5. What does `fail-fast: false` do in a matrix strategy?

- A) Prevents the workflow from failing at all
- B) Allows all matrix jobs to complete even if one fails ✅
- C) Forces jobs to run sequentially
- D) Disables error reporting

Q6. Steps inside a job share the same:

- A) GitHub token
- B) Environment variables from other jobs
- C) Runner workspace and filesystem ✅
- D) Secrets from all environments

Q7. A `push` trigger with `paths: ['src/**']` fires when:

- A) Any file in the repo changes
- B) Only files inside `src/` change ✅
- C) The `src` branch is pushed
- D) A PR targeting `src` is opened

Q8. Which trigger allows an external service to start a workflow via the GitHub REST API?

- A) `workflow_dispatch`
- B) `external_trigger`
- C) `repository_dispatch` ✅

- D) `api_dispatch`

> ***Answers:*** *C, B, D, D, B, C, B, C*

📅 **DAY 2 — TUESDAY, MARCH 18**

Topic: Contexts, Expressions & Environment Variables

Exam weight: ~10 % | Project focus: Project 1 continued

Core Concepts

1. Expressions

Expressions use `${{ }}` syntax and can appear in most workflow fields.

```
- name: Print branch
  run: echo "Branch is ${{ github.ref_name }}"

- name: Conditional step
  if: ${{ github.event_name == 'push' }}
  run: echo "Triggered by push"
```

2. Key Contexts

Context	Common properties
`github`	`github.event_name`, `github.ref`, `github.sha`, `github.actor`, `github.repository`
`env`	Variables set with `env:`
`secrets`	`secrets.MY_SECRET`
`vars`	Repository / org variables (non-sensitive)
`jobs`	`jobs.<id>.result` (in `if` conditions)
`steps`	`steps.<id>.outputs.<name>`, `steps.<id>.outcome`
`runner`	`runner.os`, `runner.arch`
`matrix`	`matrix.node-version` (inside matrix jobs)

3. Environment Variables

Scope levels (narrow wins):

```
env:                          # workflow-level
  APP_NAME: my-app

jobs:
  build:
    env:                      # job-level
      BUILD_ENV: production
    steps:
      - run: echo $APP_NAME
        env:                  # step-level (overrides abov
e)
          APP_NAME: override
```

Setting variables dynamically via `$GITHUB_ENV`:

```
- name: Set version
  run: echo "VERSION=1.2.3" >> $GITHUB_ENV

- name: Use version
  run: echo "Building ${{ env.VERSION }}"
```

4. Default Environment Variables

GitHub injects these automatically:

Variable	Value
`GITHUB_SHA`	Commit SHA that triggered the workflow
`GITHUB_REF`	Branch/tag ref (`refs/heads/main`)
`GITHUB_ACTOR`	Username who triggered the workflow
`GITHUB_REPOSITORY`	`owner/repo`
`GITHUB_WORKSPACE`	Path to the checked-out repository
`RUNNER_OS`	`Linux` , `Windows` , or `macOS`

5. Conditional Execution with `if`

```
- name: Deploy to production
  if: github.ref == 'refs/heads/main' && github.event_name
== 'push'
  run: ./deploy.sh
```

```yaml
- name: Notify on failure
  if: failure()          # built-in status function
  run: ./notify.sh

- name: Always clean up
  if: always()
  run: ./cleanup.sh
```

Status check functions: `success()`, `failure()`, `cancelled()`, `always()`

6. Outputs Between Steps & Jobs

```yaml
jobs:
  build:
    outputs:
      version: ${{ steps.get-version.outputs.version }}
    steps:
      - id: get-version
        run: echo "version=1.2.3" >> $GITHUB_OUTPUT

  deploy:
    needs: build
    steps:
      - run: echo "Deploying ${{ needs.build.outputs.versio
n }}"
```

📋 Day 2 Quiz

Q1. How do you reference a secret named `API_KEY` in an expression?

- A) `${{ env.API_KEY }}`
- B) `${{ secrets.API_KEY }}` ✅
- C) `${{ github.secrets.API_KEY }}`
- D) `${{ inputs.API_KEY }}`

Q2. Which file do you append to in order to pass an environment variable from one step to the next?

- A) `$GITHUB_OUTPUT`
- B) `$GITHUB_ENV` ✅
- C) `$GITHUB_PATH`

- D) `$GITHUB_STATE`

Q3. A step should only run when the previous step failed. Which `if` condition is correct?

- A) `if: success()`
- B) `if: always()`
- C) `if: failure()` ✅
- D) `if: error()`

Q4. Which context holds the operating system of the runner?

- A) `github.os`
- B) `runner.os` ✅
- C) `env.RUNNER_OS`
- D) `matrix.os`

Q5. `$GITHUB_OUTPUT` is used to:

- A) Print log output
- B) Set step outputs that can be referenced by later steps/jobs ✅
- C) Export environment variables
- D) Upload artifacts

Q6. A job uses `needs: [build, test]`. Which context gives access to the `build` job's outputs?

- A) `steps.build.outputs`
- B) `jobs.build.outputs`
- C) `needs.build.outputs` ✅
- D) `env.build.outputs`

Q7. The `vars` context is used for:

- A) Encrypted secrets
- B) Non-sensitive repository or organisation configuration variables ✅
- C) Matrix variable substitution
- D) Runner environment variables

Q8. Which expression evaluates to `true` on a push to `main`?

- A) `github.ref == 'main'`

- B) `github.ref == 'refs/heads/main'` ✅
- C) `github.branch == 'main'`
- D) `github.event.ref == 'main'`

> ***Answers:*** *B, B, C, B, B, C, B, B*

📅 **DAY 3 — WEDNESDAY, MARCH 19**

Topic: Runners & Execution Environments

Exam weight: ~10 % | Project focus: Project 2

Core Concepts

1. GitHub-Hosted Runners

Pre-provisioned virtual machines managed by GitHub.

Label	OS	Architecture
`ubuntu-latest`	Ubuntu 22.04	x64
`ubuntu-22.04`	Ubuntu 22.04	x64
`ubuntu-20.04`	Ubuntu 20.04	x64
`windows-latest`	Windows Server 2022	x64
`macos-latest`	macOS 14 (Sonoma)	arm64
`macos-13`	macOS 13 (Ventura)	x64

- Each job gets a **fresh, clean VM** — no state carries over between jobs.
- Pre-installed tools: git, Node.js, Python, Java, Docker, etc.
- Storage: ~14 GB SSD. Memory: 7 GB RAM. CPU: 2 cores (standard).

2. Self-Hosted Runners

Machines you register and manage yourself (on-prem, cloud VM, container).

```
jobs:
  build:
```

```
    runs-on: self-hosted          # any self-hosted runner
    # OR
    runs-on: [self-hosted, linux, x64, gpu]   # label filte
ring
```

Registration:

```
# Download runner agent from GitHub → Settings → Actions →
Runners
./config.sh --url https://github.com/ORG/REPO --token TOKEN
./run.sh
```

Self-hosted vs GitHub-hosted:

	GitHub-hosted	Self-hosted
Setup	Zero	Must install/maintain
Cost	Included (limits apply)	Infrastructure cost
Internet	Always	You control
Persistence	Ephemeral	Can be persistent
Security isolation	Strong (fresh VM)	Your responsibility

3. Runner Groups

Organise self-hosted runners into groups and restrict which repositories can use them (available at organisation/enterprise level).

4. Larger Runners

GitHub-hosted machines with more resources (4/8/16/32/64 cores, up to 64 GB RAM) — available on paid plans.

5. Container Jobs

Run a job inside a Docker container instead of directly on the runner OS:

```
jobs:
  build:
    runs-on: ubuntu-latest
    container:
      image: node:22-alpine
      credentials:
        username: ${{ secrets.DOCKER_USER }}
        password: ${{ secrets.DOCKER_PASS }}
```

```yaml
    steps:
      - run: node --version    # runs inside the container
```

6. Service Containers

Spin up sidecar services (databases, caches) for integration tests:

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
```

📋 Day 3 Quiz

Q1. Which runner label runs a job on the latest Ubuntu image maintained by GitHub?

- A) `ubuntu`
- B) `linux-latest`
- C) `ubuntu-latest` ✅
- D) `github-ubuntu`

Q2. Self-hosted runners are most appropriate when you need:

- A) Zero setup time
- B) Access to proprietary internal network resources ✅
- C) GitHub to manage software installation
- D) Maximum security isolation

Q3. A job configured with `runs-on: [self-hosted, gpu]` will use:

- A) Any self-hosted runner
- B) A GitHub-hosted GPU runner

- C) Only self-hosted runners that have the `gpu` label ✅
- D) A Docker container with GPU support

Q4. What happens to the GitHub-hosted runner environment after a job completes?

- A) It is kept for the next job in the same workflow
- B) It is archived to storage
- C) It is destroyed and a fresh VM is provisioned for each job ✅
- D) It is returned to a pool for reuse

Q5. Service containers in GitHub Actions are used to:

- A) Deploy applications
- B) Provide sidecar services like databases for integration tests ✅
- C) Cache build artifacts
- D) Run jobs in parallel

Q6. The `container:` key in a job definition makes the job steps run:

- A) On a separate GitHub-hosted runner
- B) Inside the specified Docker container on the runner ✅
- C) In a VM snapshot
- D) Inside GitHub Codespaces

Q7. Runner Groups (at the organisation level) are used to:

- A) Group workflow jobs for display purposes
- B) Restrict which repositories can use specific self-hosted runners ✅
- C) Run jobs in parallel
- D) Share artifacts between runs

Q8. Which GitHub-hosted runner uses an ARM64 architecture?

- A) `ubuntu-latest`
- B) `windows-latest`
- C) `macos-13`
- D) `macos-latest` ✅

> ***Answers:*** *C, B, C, C, B, B, B, D*

📅 **DAY 4 — THURSDAY, MARCH 20**

Topic: Artifacts, Caching & Dependencies

Exam weight: ~10 % | Project focus: Project 1 — Artifacts section

Core Concepts

1. Artifacts

Artifacts persist files from a workflow run so they can be downloaded later (build outputs, test reports, binaries).

```yaml
- name: Build application
  run: npm run build

- name: Upload build artifact
  uses: actions/upload-artifact@v4
  with:
    name: build-output          # artifact name
    path: dist/                 # files/folders to upload
    retention-days: 5           # default 90 days (max 90)
    if-no-files-found: error    # error | warn | ignore
```

Downloading in the same workflow run:

```yaml
- name: Download build artifact
  uses: actions/download-artifact@v4
  with:
    name: build-output
    path: downloaded-dist/
```

Downloading across workflow runs → use the GitHub API or `actions/download-artifact` with `run-id`.

2. Caching Dependencies

Cache reduces install time by storing and restoring node_modules / pip cache / Maven local repo between runs.

```yaml
- name: Cache npm packages
  uses: actions/cache@v4
  with:
    path: ~/.npm                        # what to cache
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-loc
k.json') }}
```

```
    restore-keys: |                    # fallback keys (pr
efix match)
      ${{ runner.os }}-npm-
```

How the cache key works:

1. Exact match on `key` → cache restored, cache step shows "Cache hit".
2. No exact match → tries `restore-keys` (prefix search, most recent match).
3. No match at all → cache miss, job runs without cache.
4. After the job, if `key` is new, the cache is saved.

3. `setup-node` Built-in Caching

`actions/setup-node@v4` can cache npm/yarn/pnpm automatically:

```
- uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: 'npm'    # or 'yarn' or 'pnpm'
```

This is equivalent to using `actions/cache` manually with npm defaults.

4. Cache Limits & Scope

- **Size limit:** 10 GB per repository.
- **Eviction:** LRU — least recently accessed caches are evicted first.
- **Scope:** Caches are isolated per branch. A PR job can access caches from the **base branch** (e.g., `main`) but not from other PRs.
- **Caches are not shared** across repositories.

5. Artifacts vs Cache

	Artifacts	Cache
Purpose	Store build outputs for download	Speed up repeated steps
Access	Any run (via API or UI)	Same or future runs on same branch

Retention	1–90 days (configurable)	Up to 7 days (reset on use)
Typical use	Test reports, binaries, logs	`node_modules`, pip virtualenvs

---

📋 *Day 4 Quiz*

Q1. Which action uploads files produced during a workflow run?

- A) `actions/cache@v4`
- B) `actions/upload-artifact@v4` ✅
- C) `actions/store-artifact@v4`
- D) `actions/save-output@v4`

Q2. The maximum retention period for artifacts is:

- A) 7 days
- B) 30 days
- C) 90 days ✅
- D) 1 year

Q3. A cache `restore-keys` list provides:

- A) Fallback prefix-match keys when the exact `key` is not found ✅
- B) Secondary encryption keys
- C) Keys to restore artifacts
- D) Alternate branch names to search

Q4. The `key` for a cache should include `hashFiles('**/package-lock.json')` because:

- A) It encrypts the cache
- B) It ensures the cache is invalidated when dependencies change ✅
- C) It compresses the cache
- D) It is required by GitHub Actions

Q5. What is the maximum cache storage size per repository?

- A) 1 GB
- B) 5 GB
- C) 10 GB ✅
- D) 50 GB

Q6. Can a pull request workflow job access caches created by the default branch?

- A) No, caches are completely isolated
- B) Yes, PRs can read caches from the base branch ✅
- C) Only if the cache was shared explicitly
- D) Only for private repositories

Q7. What is the main difference between artifacts and cache?

- A) Artifacts are faster than cache
- B) Cache is for storing build outputs; artifacts are for speeding up installs
- C) Artifacts are for sharing between runs/download; cache speeds up repeated steps ✅
- D) There is no difference

Q8. `actions/setup-node@v4` with `cache: 'npm'` is equivalent to:

- A) Running `npm install` automatically
- B) Using `actions/cache@v4` with npm-specific path and key conventions ✅
- C) Uploading node_modules as an artifact
- D) Setting `NODE_ENV=production`

> ***Answers:*** *B, C, A, B, C, B, C, B*

📅 **DAY 5 — FRIDAY, MARCH 21**

Topic: Reusable Workflows & Composite Actions

Exam weight: ~15 % | Project focus: Project 3

Core Concepts

1. Reusable Workflows

A workflow file can be called by another workflow, enabling DRY (Don't Repeat Yourself) pipelines.

Called workflow (`.github/workflows/reusable-test.yml`):

```yaml
on:
  workflow_call:            # makes this workflow callable
    inputs:
      node-version:
        required: true
        type: string
    secrets:
      NPM_TOKEN:
        required: false
    outputs:
      test-result:
        description: "Result of the test run"
        value: ${{ jobs.test.outputs.result }}

jobs:
  test:
    runs-on: ubuntu-latest
    outputs:
      result: ${{ steps.run-tests.outputs.result }}
    steps:
      - uses: actions/checkout@v4
      - id: run-tests
        run: |
          npm test
          echo "result=passed" >> $GITHUB_OUTPUT
```

Caller workflow:

```yaml
jobs:
  call-tests:
    uses: ./.github/workflows/reusable-test.yml
    with:
      node-version: '22'
    secrets:
      NPM_TOKEN: ${{ secrets.NPM_TOKEN }}
    # OR: secrets: inherit   (passes ALL caller secrets)
```

Key rules:

- A reusable workflow must be triggered by `workflow_call`.
- The `uses:` path is `owner/repo/.github/workflows/file.yml@ref` or `./.github/workflows/file.yml` for the same repo.
- Caller and callee share the same runner billing context.
- Up to **20 levels** of nesting allowed.
- A caller job that uses `uses:` cannot also define `steps:`.

2. Composite Actions

An action that groups multiple steps (shell commands, other actions) into a single reusable action.

`.github/actions/setup-and-test/action.yml`:

```yaml
name: 'Setup and Test'
description: 'Install deps and run tests'
inputs:
  node-version:
    description: 'Node.js version'
    required: true
    default: '22'
outputs:
  test-result:
    description: 'Test pass/fail'
    value: ${{ steps.test.outputs.result }}

runs:
  using: composite
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
    - id: test
      shell: bash
      run: |
        npm ci
        npm test
        echo "result=passed" >> $GITHUB_OUTPUT
```

Usage in a workflow:

```yaml
- uses: ./.github/actions/setup-and-test
  with:
    node-version: '22'
```

3. Reusable Workflow vs Composite Action

	Reusable Workflow	Composite Action
Unit	Entire workflow (jobs + steps)	Steps only
Can define jobs	✅ Yes	❌ No
Can use `runs-on`	✅ Yes	❌ Inherits caller's runner

Inputs	`inputs:` + `secrets:` blocks	`inputs:` block
Used with	`uses:` at job level	`uses:` at step level
Secrets passing	Explicit or `secrets: inherit`	Via `secrets` context in calling job

4. JavaScript & Docker Container Actions

JavaScript action (`action.yml`):

```
runs:
  using: node20
  main: dist/index.js
```

Docker container action (`action.yml`):

```
runs:
  using: docker
  image: Dockerfile
  args:
    - ${{ inputs.my-input }}
```

5. Action Metadata (`action.yml`)

Every action must have an `action.yml` (or `action.yaml`) at its root with:

- `name` , `description` , `author`
- `inputs` (with `description` , `required` , `default`)
- `outputs` (with `description` , `value`)
- `runs` (defines the runtime)

📋 *Day 5 Quiz*

Q1. What event must a reusable workflow respond to?

- A) `workflow_run`
- B) `workflow_dispatch`
- C) `workflow_call` ✅

- D) `repository_dispatch`

Q2. A composite action is defined with:

- A) `runs: using: composite` ✅
- B) `runs: using: steps`
- C) `type: composite`
- D) `action-type: composite`

Q3. A job that calls a reusable workflow with `uses:` can also define `steps:` in the same job.

- A) True
- B) False ✅

Q4. To pass ALL secrets from the caller to a reusable workflow, use:

- A) `secrets: all`
- B) `secrets: pass-through`
- C) `secrets: inherit` ✅
- D) `secrets: forward`

Q5. What file must every custom action contain?

- A) `workflow.yml`
- B) `action.yml` ✅
- C) `Dockerfile`
- D) `index.js`

Q6. A reusable workflow (called with `uses:`) can define its own `runs-on:` for each job.

- A) True ✅
- B) False

Q7. Composite actions can contain:

- A) Job definitions with their own runners
- B) Only shell commands
- C) Steps, including calls to other actions ✅
- D) Full workflow definitions

Q8. The maximum nesting depth for reusable workflows is:

- A) 3

- B) 5
- C) 10
- D) 20 ✅

> ***Answers:*** *C, A, B, C, B, A, C, D*

📅 DAY 6 — SATURDAY, MARCH 22

Topic: Secrets, Security, Deployment Environments & Concurrency

Exam weight: ~15 % | Project focus: Project 2

Core Concepts

1. Secrets

Encrypted key-value pairs; values are **never** exposed in logs (masked).

Scopes:

- **Repository** secrets → `Settings > Secrets and variables > Actions`
- **Environment** secrets → override repository secrets for a specific environment
- **Organisation** secrets → shared across multiple repositories (with access policies)

```
steps:
  - run: ./deploy.sh
    env:
      API_TOKEN: ${{ secrets.API_TOKEN }}    # access via e
nv var
```

Rules:

- Secrets are not passed to workflows triggered by a **forked repository** (unless you explicitly allow it).
- Maximum 48 KB per secret value.
- Organisation-level secrets can be limited to specific repos.

2. Variables (non-sensitive)

```
- run: echo ${{ vars.APP_NAME }}
```

Repository/org-level configuration variables (not encrypted, visible in logs).

3. Environments

Environments let you add **deployment protection rules** and environment-specific secrets.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://myapp.com    # shown in GitHub UI
    steps:
      - run: ./deploy.sh
        env:
          PROD_KEY: ${{ secrets.PROD_KEY }}    # env-level
secret
```

Protection rules:

- **Required reviewers** — one or more people must approve before the job runs.
- **Wait timer** — delay before the job starts (0–43,200 minutes).
- **Deployment branches** — restrict which branches can deploy.
- **Custom deployment protection rules** (GitHub Apps).

4. Permissions (`permissions`)

Restrict the `GITHUB_TOKEN` permissions to principle of least privilege:

```
permissions:
  contents: read        # read repo files
  packages: write       # publish packages
  id-token: write       # OIDC token for cloud auth
  pull-requests: write  # comment on PRs
```

Can be set at **workflow level** or **job level** (job-level overrides workflow-level).

Default permissions can be set to **read-all** or **write-all** in the repo/org settings.

5. GITHUB_TOKEN

Automatically created per workflow run; used to authenticate against the GitHub API.

```
- name: Create release
  uses: actions/create-release@v1
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

6. OpenID Connect (OIDC)

Obtain short-lived cloud credentials (AWS, Azure, GCP) **without storing long-lived secrets**.

```
permissions:
  id-token: write
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789:role/github-ac
tions
      aws-region: us-east-1
```

7. Concurrency Control

Prevent multiple deployments running simultaneously:

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true    # cancel older run if new one
starts
```

📋 *Day 6 Quiz*

Q1. Which type of secret takes priority over a repository-level secret with the same name?

- A) Organisation secret
- B) Environment secret ✅

- C) Workflow-level secret
- D) Job-level secret

Q2. The `GITHUB_TOKEN` secret is:

- A) Manually created by an admin
- B) Automatically generated for each workflow run ✅
- C) Shared across all repositories
- D) Valid for 30 days

Q3. A required reviewer on a deployment environment means:

- A) The code must pass a code review before merging
- B) A designated person must approve the deployment job before it runs ✅
- C) The PR must have an approved review
- D) The job is skipped until approved

Q4. To give a workflow the minimum permissions needed, you should:

- A) Use `permissions: write-all`
- B) Delete the GITHUB_TOKEN
- C) Use `permissions:` with only the specific permissions needed ✅
- D) Set `secrets: readonly: true`

Q5. OIDC (OpenID Connect) in GitHub Actions allows you to:

- A) Log in to GitHub
- B) Authenticate with cloud providers without storing long-lived credentials ✅
- C) Enable two-factor authentication for workflows
- D) Encrypt workflow secrets

Q6. `concurrency: cancel-in-progress: true` will:

- A) Queue multiple runs
- B) Wait for the in-progress run to complete
- C) Cancel the currently running workflow when a new one starts ✅
- D) Prevent new runs from starting

Q7. Secrets are accessible in workflows triggered by forks:

- A) Always
- B) Never
- C) Only if the repository setting explicitly allows it ✅

- D) Only for public repositories

Q8. Which `permissions` key is required to request an OIDC token?

- A) `contents: write`
- B) `packages: write`
- C) `deployments: write`
- D) `id-token: write` ✅

> ***Answers:*** *B, B, B, C, B, C, C, D*

📅 **DAY 7 — SUNDAY, MARCH 23**

Topic: Final Review — Monitoring, Troubleshooting & Enterprise Management

Exam date tomorrow! 🎯 | All three projects

Core Concepts

1. Workflow Run Status

- **Queued** → waiting for a runner
- **In progress** → running
- **Completed** → finished (success / failure / cancelled / skipped)

2. Debugging Workflows

Enable debug logging by setting a repository secret:

```
ACTIONS_RUNNER_DEBUG = true    # verbose runner diagnostics
ACTIONS_STEP_DEBUG = true      # verbose step-level output
```

`tmate` **/ interactive debugging:**

```yaml
- uses: mxschmitt/action-tmate@v3
  if: failure()    # only open SSH session on failure
```

3. Skipping Workflows

Include `[skip ci]`, `[ci skip]`, `[no ci]`, or `[skip actions]` in a commit message to skip push/PR triggered workflows.

4. Re-running Workflows

- Re-run all jobs, or re-run failed jobs only.
- Jobs can optionally be re-run with **debug logging enabled**.

5. Enterprise / Organisation Management

Feature	Description
Allowed actions policy	Restrict which actions can be used (all / local only / select)
Runner groups	Assign self-hosted runners to specific repos or access policies
Required workflows	Enforce a workflow to run on all repos in an org
Audit log	Track workflow runs, secret access, runner registrations
IP allow lists	Restrict which IPs can trigger workflows
GITHUB_TOKEN permissions	Set org-wide default (read-all vs write-all)

6. Workflow Billing

- Public repositories: **free unlimited minutes**
- Private repositories: minutes per billing plan; GitHub-hosted runners multiply minutes by OS factor:
  - Linux: × 1
  - Windows: × 2
  - macOS: × 10

7. Status Badges

Add a workflow status badge to `README.md`:

```
![CI](https://github.com/OWNER/REPO/actions/workflows/ci.yml/badge.svg)
```

8. Dependabot & Actions Security

- **Dependabot** can automatically update `actions/checkout@v3` → `@v4`.
- **Pin actions to SHA** for maximum security:

```
uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4.2.2
```

- Use `third-party-action-audit` or Dependabot alerts for vulnerable actions.

9. Workflow Limits

Limit	Value
Max workflow run time	35 days
Max job execution time	6 hours
API requests per hour (GITHUB_TOKEN)	1,000
Concurrent jobs (free)	20 (Linux), 5 (macOS)
Artifacts + cache storage	500 MB – 50 GB (plan dependent)

📋 Day 7 — Final Mixed Quiz (30 Questions)

Q1. Which commit message skip keyword is recognised by GitHub Actions?

- A) `[no-test]`
- B) `[skip ci]` ✅
- C) `[bypass]`
- D) `[ignore actions]`

Q2. Which secret enables verbose runner diagnostics?

- A) `ACTIONS_DEBUG=true`
- B) `RUNNER_DEBUG=true`
- C) `ACTIONS_RUNNER_DEBUG=true` ✅
- D) `GITHUB_DEBUG=true`

Q3. A macOS GitHub-hosted runner minute multiplier is:

- A) 1
- B) 2
- C) 5
- D) 10 ✅

Q4. Maximum single job execution time is:

- A) 1 hour
- B) 6 hours ✅
- C) 24 hours
- D) 35 days

Q5. To restrict which third-party actions can be used in an organisation, you configure:

- A) Branch protection rules
- B) Allowed actions policy ✅
- C) CODEOWNERS file
- D) Required status checks

Q6. `secrets: inherit` in a reusable workflow call passes:

- A) Only `GITHUB_TOKEN`
- B) Only explicitly listed secrets
- C) All secrets available to the calling job ✅
- D) Organisation-level secrets only

Q7. A job's `if: always()` condition means:

- A) The job only runs on success
- B) The job runs regardless of the outcome of previous jobs ✅
- C) The job runs in every environment
- D) The job ignores secrets

Q8. Which action is used to check out the repository code?

- A) `actions/setup-node@v4`
- B) `actions/checkout@v4` ✅
- C) `actions/clone@v4`
- D) `actions/fetch@v4`

Q9. `npm ci` differs from `npm install` in that it:

- A) Installs global packages

- B) Uses `package-lock.json` for exact versions and fails if the lock file is out of date ✅
- C) Skips dev dependencies
- D) Caches packages automatically

Q10. A workflow is triggered by `schedule: cron: '0 0 * * *'`. This means:

- A) Every hour
- B) Every day at midnight UTC ✅
- C) Every Sunday
- D) Every minute

Q11. To prevent two deployments from running at the same time, you use:

- A) `mutex:`
- B) `lock:`
- C) `concurrency:` ✅
- D) `serialize:`

Q12. The `GITHUB_WORKSPACE` environment variable holds:

- A) The runner's home directory
- B) The GitHub API URL
- C) The path to the checked-out repository ✅
- D) The workflow file path

Q13. Which output file is used to pass a value from one step to a later step?

- A) `$GITHUB_ENV`
- B) `$GITHUB_OUTPUT` ✅
- C) `$GITHUB_PATH`
- D) `$GITHUB_SUMMARY`

Q14. Forked repositories cannot access secrets by default because:

- A) GitHub limits all secrets to private repos
- B) An untrusted fork could expose secrets to the fork maintainer ✅
- C) Forks use a different `GITHUB_TOKEN`
- D) Secrets expire for forks

Q15. To write a step summary (visible in the workflow run UI), append to:

- A) `$GITHUB_OUTPUT`

- B) `$GITHUB_ENV`
- C) `$GITHUB_STEP_SUMMARY` ✅
- D) `$GITHUB_SUMMARY`

Q16. The maximum number of reusable workflow nesting levels is:

- A) 3
- B) 5
- C) 10
- D) 20 ✅

Q17. Which `permissions` entry allows writing comments to a pull request?

- A) `issues: write`
- B) `contents: write`
- C) `pull-requests: write` ✅
- D) `checks: write`

Q18. An action is pinned to a commit SHA instead of a tag for what reason?

- A) SHA pinning is required by GitHub
- B) SHA pins prevent a tag from being silently redirected to malicious code ✅
- C) Tags do not work with composite actions
- D) Commit SHAs are faster to resolve

Q19. A `wait-timer` on a deployment environment introduces:

- A) A retry delay on failure
- B) A mandatory pause before a deployment job starts ✅
- C) A timeout for the deployment
- D) A cooldown period after deployment

Q20. The `$GITHUB_PATH` file is used to:

- A) Set the working directory
- B) Add a directory to the runner's `PATH` for subsequent steps ✅
- C) Store artifact paths
- D) Define the checkout path

Q21. Which of the following is NOT a valid `workflow_dispatch` input type?

- A) `string`
- B) `boolean`
- C) `number` ✅ (not supported — use `string`)
- D) `choice`

Q22. `actions/cache@v4` with a key that already exists will:

- A) Overwrite the existing cache
- B) Fail the step
- C) Skip saving but still restore ✅
- D) Create a duplicate cache entry

Q23. The `if: github.event_name == 'push' && github.ref == 'refs/heads/main'` condition ensures a step runs:

- A) On any push
- B) Only on pull requests to main
- C) Only when pushing commits directly to main ✅
- D) On any event targeting main

Q24. Docker container actions (`runs: using: docker`) require the runner to:

- A) Be a macOS machine
- B) Have Docker installed ✅
- C) Use a self-hosted runner
- D) Have root access

Q25. An organisation's "Required workflows" feature:

- A) Forces specific branch protection rules
- B) Enforces a workflow to run across all repositories in the organisation ✅
- C) Requires all workflows to pass before merging
- D) Mandates workflow approvals

Q26. `hashFiles('**/package-lock.json')` in a cache key:

- A) Hashes the runner's file system
- B) Produces a hash of all matching files, changing when dependencies change ✅
- C) Encrypts the cache

- D) Lists all lock files

Q27. Which action is commonly used to publish a package to GitHub Packages?

- A) `actions/publish@v4`
- B) `actions/upload-artifact@v4`
- C) `actions/setup-node@v4` with `registry-url` + `npm publish` ✅
- D) `actions/release@v4`

Q28. A job using `environment: production` with required reviewers will:

- A) Run immediately
- B) Be cancelled if the reviewer does not respond in time
- C) Pause and wait for approval before the job's steps execute ✅
- D) Send a notification only

Q29. What is the purpose of the `outputs:` block in a `workflow_call` workflow?

- A) To define environment variables
- B) To pass values back to the calling workflow ✅
- C) To specify artifact paths
- D) To set step outputs

Q30. The GitHub Actions billing multiplier for a Windows runner is:

- A) 1
- B) 2 ✅
- C) 5
- D) 10

> ***Answers:***
> *B,C,D,B,B,C,B,B,B,B,C,C,B,B,C,D,C,B,B,B,C,C,C,B,B,B,C,C,B,B*

🏁 **EXAM DAY — MONDAY, MARCH 24**

Last-Minute Checklist

- ☐ Review the three project workflow files in this repository

- ☐ Skim the Day 7 table of limits
- ☐ Remember: `workflow_call` = reusable, `workflow_dispatch` = manual
- ☐ `$GITHUB_ENV` → env vars | `$GITHUB_OUTPUT` → step outputs | `$GITHUB_STEP_SUMMARY` → UI summary
- ☐ Artifacts = download later | Cache = speed up repeated runs
- ☐ Environment secrets > Repository secrets > Organisation secrets (priority)
- ☐ macOS billing multiplier = 10, Windows = 2, Linux = 1
- ☐ SHA pinning > tag pinning (security)
- ☐ `concurrency:` prevents duplicate deployments
- ☐ OIDC = `id-token: write` permission

Good luck! You've got this! 🚀

📚 **QUICK REFERENCE CARD**

```
EVENTS:        push | pull_request | workflow_dispatch | schedu
le | workflow_call
               release | repository_dispatch

CONTEXTS:      github.* | env.* | secrets.* | vars.* | steps.*
| jobs.*
               runner.* | matrix.* | needs.*

STATUS FNs:    success() | failure() | cancelled() | always()

SPECIAL FILES: $GITHUB_ENV        → set env vars for next steps
               $GITHUB_OUTPUT     → set step outputs
               $GITHUB_STEP_SUMMARY → write to job summary UI
               $GITHUB_PATH       → add to PATH

BILLING:       Linux×1  Windows×2  macOS×10

LIMITS:        Job: 6h | Run: 35 days | Cache: 10GB/repo | Nest
ing: 20 levels
```